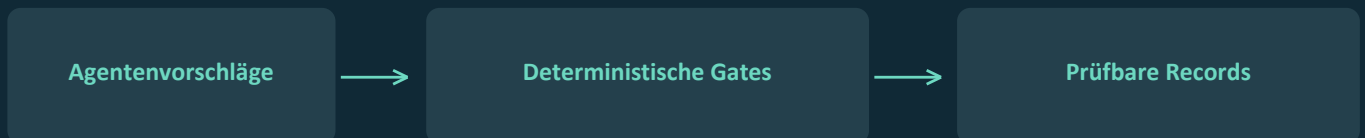


ROLLEN, VERTRÄGE UND BETRIEBSNACHWEISE

Stochastische Agenten, deterministische Kontrolle.

Wie getrennte Rollen, gebundene Nachweise und eine prüfbare Historie agentische Entwicklungsarbeit kontrollierbar machen.



DOKUMENTIERTER STAND

Phase A: Der Rollenvertrag ist vom Provider entkoppelt. Die aktuelle Besetzung ist praktisch erprobt; weitere Kombinationen haben einen definierten Qualifikationspfad.

Dieter Esser

Technisches Whitepaper · Ausgabe 1.3 · 28. September 2026

01

ZUSAMMENFASSUNG

Was Phase A erreicht

Ein Architekturbericht über einen real betriebenen Entwicklungsorchestrator und die Grenzen seines heutigen Nachweises.

Der Dual-Agent-Orchestrator organisiert Softwareänderungen als kontrollierten Prozess: Ein Implementer plant und ändert Code, ein davon getrennter Reviewer beurteilt Plan und Umsetzung. Der Orchestrator führt die deterministische Validierung aus, bindet ihre Ergebnisse an den geprüften Stand und entscheidet anhand expliziter Regeln, ob der Prozess fortgesetzt werden darf. Sprachmodelle liefern Vorschläge und Urteile; die Autorität über Zustandsübergänge bleibt im Programm. [1]

Phase A beseitigt die frühere Gleichsetzung von Rolle und Anbieter. **Implementer** und **Reviewer** sind jetzt fachliche Rollen; Provider, Modell, Aufwand und Laufzeitgrenze gehören zu gebundenen Profilen. Drei konfigurierbare Slots bilden Planung/Implementierung, laufendes Review und abschließendes Branchreview ab. Der dritte Slot ist eine gesonderte Reviewer-Aufgabe, keine dritte fachliche Rolle. [1, 2]

DIE ZENTRALE AUSSAGE

Die Architektur erlaubt, weitere Provider systematisch anzuschließen. Heute zugelassen und erprobt ist Codex / Claude / Claude. Eine neutrale Schnittstelle allein erteilt noch keiner anderen Kombination eine Freigabe.

Diese Ausgabe dokumentiert den abgenommenen Stand der Phase A vom 28. September 2026. Der unterstützte Betriebsrahmen ist Linux beziehungsweise WSL 2; die Fallstudie wurde unter WSL 2 / Ubuntu durchgeführt. [1, 3]

Das Papier verbindet Implementierung und Konfiguration, gespeicherte Tests und Laufrecords sowie Erfahrungen aus Entwicklungs- und Anwendungseinsätzen. Die dokumentierte Abnahme macht den Ablauf an einer realen Aufgabe nachvollziehbar. Weitergehende Aussagen zur Zuverlässigkeit erfordern eine breitere Messreihe mit unterschiedlichen Aufgaben und Fehlersituationen.

LESEWEG

Prinzip und Architektur S. 3-5 · Provider und JSON S. 6-7
Records und Recovery S. 8-9 · Praxis und Messwerte S. 10-11
Abo-Betrieb S. 12 · Einordnung und Quellen S. 13-14

02

ENTWURFSPRINZIP

Warum Schranken nötig sind

Die Variabilität eines Modells ist nützlich beim Entwerfen. Als alleinige Grundlage für einen Workflowzustand ist sie zu schwach.

Ein LLM kann eine gute Lösung finden und trotzdem ein Feld auslassen, einen offenen Befund falsch referenzieren oder eine gelungene Änderung zu früh als fertig melden. Auch zwei plausible Antworten können sich widersprechen. Die Kontrolllogik muss deshalb auf prüfbaren Bedingungen beruhen: Wer antwortet auf welchen Auftrag, für welchen Codezustand und mit welcher Befugnis?

Die Entwicklung begann mit Kommunikation über Markdown-Dateien. Deren Ausgabeform erwies sich als zu unstet für verlässliche automatische Auswertung. Die JSON-Schicht war damit keine kosmetische Verbesserung. Sie schuf die Voraussetzung, strukturierte Entscheidungen, eindeutige Bindungen und negative Antworten maschinell auszuwerten. Markdown bleibt als lesbare Dokumentation und Auditansicht sinnvoll. [1, 8]

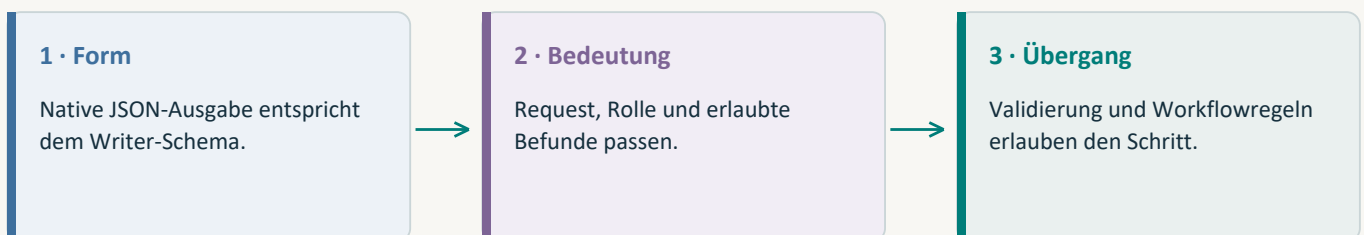


ABB. 01 Drei getrennte Prüfungen. Syntaktisch korrektes JSON ist erst der Anfang; es ist noch keine fachlich zulässige Entscheidung.

Im heutigen Vertrag werden fehlende oder ungültige native Antworten zurückgewiesen. Es gibt keinen Freitextparser als Ersatz und keine stillschweigende Reparatur zu einer Freigabe. Ein begrenzter neuer Provideraufruf kann einen Fehler behandeln; dessen Antwort muss wiederum den unveränderten Vertrag erfüllen. Die Abnahme zeigt genau diese Grenze an einer unzulässigen Finding-Referenz. [1, 3]

Deterministisch meint hier die Anwendung der Kontrollregeln auf gebundene Daten. Es meint nicht, dass Modellantworten, Tests oder die Außenwelt stets identisch ausfallen. Ein grüner Testlauf belegt die ausgeführten Prüfungen am gemessenen Stand. Seine Aussagekraft bleibt von der Testauswahl und deren Qualität abhängig.

Fehler früh abfangen

Begrenzte Slices, explizite Befunde und geprüfte Zwischenstände verkürzen die Rückkopplung. Abweichungen können dadurch früher erkannt und korrigiert werden, bevor weitere Änderungen auf ihnen aufbauen. Das senkt das Risiko schleichender Fehlerfortschreibung; die Aussagekraft bleibt von den Prüfungen abhängig. [1]

03

ARCHITEKTUR

Eine kleine Autoritätsgrenze

Vereinfachtes UML-Komponentendiagramm: Vorschläge entstehen bei den Agenten, verbindliche Prozessfakten im Orchestrator.

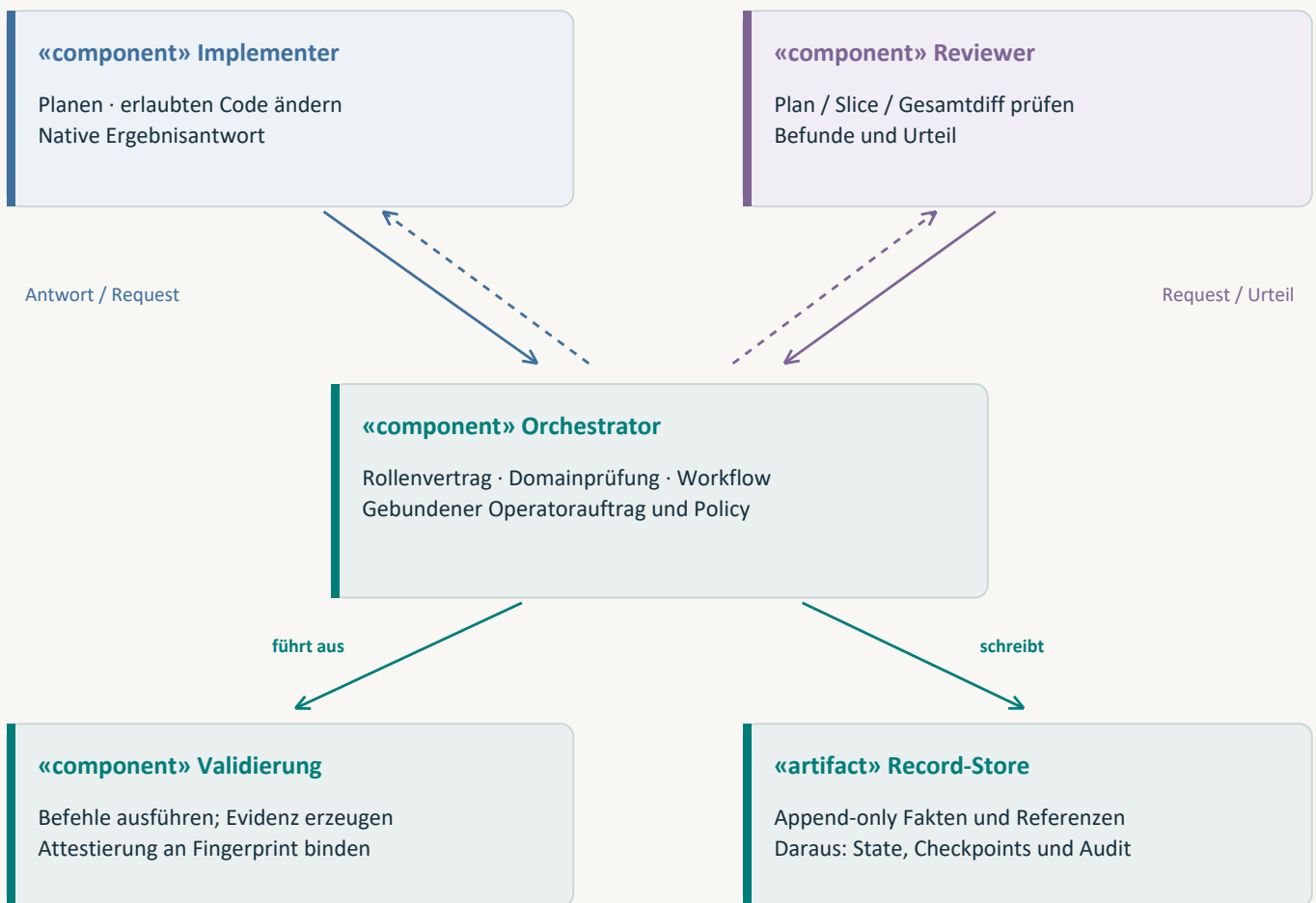


ABB. 02 Logische Zuständigkeiten, keine Darstellung einzelner Prozesse. Record-Schreibvorgänge und attestierte Validierung werden vom Orchestrator ausgeführt.

Der Implementer besitzt kein Recht zur eigenen Freigabe und schließt keine reviewer-eigenen Findings. Der Reviewer soll ausschließlich lesend arbeiten. Phase A verwendet dafür einen separaten Snapshot, entfernte Schreibbits und begrenzte CLI-Werkzeuge. Das sind konkrete Schutzmechanismen; daraus folgt noch kein universeller Nachweis einer unüberwindbaren Betriebssystemgrenze. Die breitere Isolationsprüfung steht ausdrücklich im nächsten Qualifikationspaket. [1, 2, 4]

Git-Transaktionen und Queue-Bewegungen gehören zur Steuerung, nicht zu den Agentenantworten. Ein optionaler lokaler Abschlussmerge ist eine gebundene Workflowpolicy; im dokumentierten Abnahmelauf war er deaktiviert. Der Merge erfolgte nach dem abgeschlossenen Lauf. [1, 3]

04

GEWALTENTEILUNG

Review ist ein eigener Prozess

Das Vier-Augen-Prinzip als technischer Rollenvertrag. Vereinfachtes UML-Sequenzdiagramm für Planung und Prüfung eines Slices.

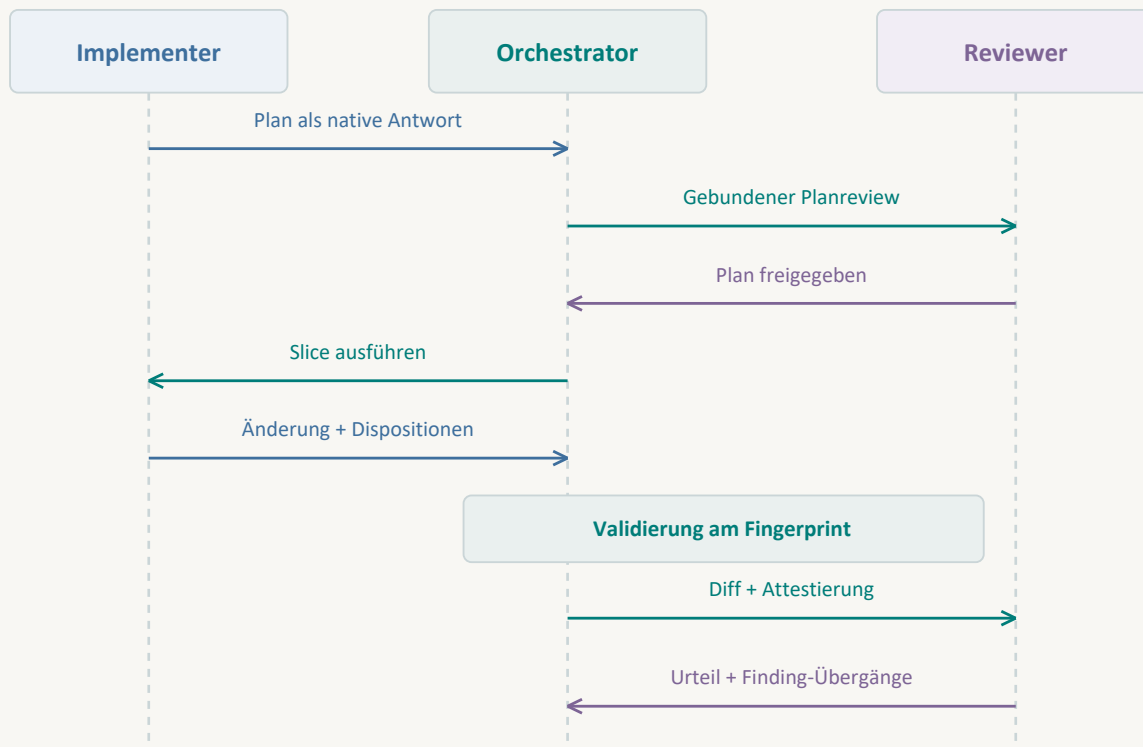


ABB. 03 Bei Ablehnung folgt eine Korrektur mit erneuter Prüfung. Erst nach erfüllten Commitregeln schreibt der Orchestrator den Slice-Commit.

Freigaberelevant sind die vom Orchestrator selbst ausgeführten Prüfungen und ihre an den Änderungsstand gebundenen Ergebnisse. Selbstauskünfte des Implementers ersetzen diesen Nachweis nicht. Auch Teständerungen müssen im Review fachlich geprüft werden. [1]

Ein Finding bleibt an seinen ursprünglichen Slice und seinen meldenden Reviewer gebunden. Der Implementer muss jeden offenen Befund begründet behandeln: akzeptieren und korrigieren oder einen gewöhnlichen Befund begründet zurückweisen. Die Schließung bleibt beim Reviewer. Ein Blocker ist nicht durch eine Ablehnung des Implementers erledigt. [1]

Die erste Prüfung dient der Entdeckung; weitere abgelehnte Runden müssen Fortschritt zeigen. Grenzen für Runden und Abschlussreviews verhindern endlose Schleifen. Nach allen Slices untersucht der Finalreviewer den vollständigen Branch-Diff. Neue Befunde führen in einen gewöhnlichen Folgeauftrag; das Abschlussreview selbst ist keine Korrekturschleife. [1]

05

PROVIDER UND ROLLEN

Heute unterstützt, morgen geprüft

Der neutralisierte Vertrag ist die Voraussetzung für Austauschbarkeit. Die konkrete Zulassung bleibt eine eigene Entscheidung.

Provider / CLI	Implementer	Reviewer	Finalreviewer
OpenAI Codex	Phase A	Phase C geplant	Phase C geplant
Anthropic Claude Code	Phase C geplant	Phase A	Phase A
Google Antigravity CLI	Phase C geplant	Phase B geplant	Phase B geplant

Phase A ist heute zugelassen; B/C sind geplant. „certified“ bezeichnet eine projektinterne Freigabe. [2, 4]
Antigravity CLI (agy) ist Googles terminalbasiertes Werkzeug für agentische Softwareentwicklung. [14]

Was im Lauf gebunden wird

Die TOML-Konfiguration ordnet jedem Slot ein Profil zu. Vor dem Start werden unter anderem Provider, Hersteller, Modellanforderung, Effort, Timeout, Binary-Identität sowie Capability-, Transport-, Rechte-, Policy- und Zulassungsdigest festgehalten. Resume prüft diese Bindungen erneut. Der Abnahmelauf verwendete Codex CLI 0.156.1 mit gpt-6-sol/high und Claude Code 2.1.283 mit opus/high. Modellnamen sind angeforderte Profilwerte, kein unabhängiger Nachweis der internen Modellausführung. [2, 3]

Die Grenze der freien Besetzung

Implementer und beide Reviewer-Slots müssen verschiedenen Herstellern zugeordnet sein. Beide Review-Slots dürfen denselben Hersteller nutzen. Nach entsprechender Qualifikation wäre etwa Claude / Codex / Codex möglich; Claude / Codex / Claude bliebe unzulässig. Andere Aliasnamen, Modelle oder Binarypfade ändern die Herstelleridentität nicht. [2, 4]

DER NÄCHSTE SCHRITT

Phase B soll Antigravity zunächst experimentell als Reviewer zulassen. Phase C erweitert die Rollenbesetzung je geprüfter Kombination und verlangt einen realen Funktionsnachweis der neuen Topologie.

06

STRUKTURIERTE KOMMUNIKATION

Gleiches JSON ist nicht genug

Die OpenSQL-Analogie hilft: Ein einheitlicher fachlicher Vertrag braucht Übersetzungen für unterschiedliche Providergrenzen.

JSON normiert eine Syntax. Es normiert weder die vom Anbieter akzeptierte Teilmenge von JSON Schema noch dessen Regex-Dialekt, Transporthülle, Größenlimits oder Werkzeugrechte. Zwei CLIs können beide strukturierte Ausgabe anbieten und denselben Writer trotzdem unterschiedlich behandeln. Ein universelles Promptformat löst diese Unterschiede nicht.

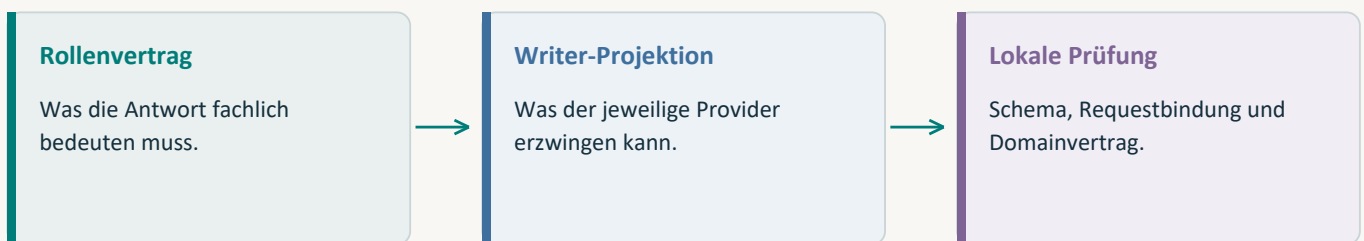


ABB. 04 Ein Vertrag, unterschiedliche Projektionen. Eine entfernte Writer-Einschränkung braucht eine begründete lokale Gegenprüfung.

Erfahrungen aus dem Umbau

In Phase A wurden fachliche Rollen und v3-Nachrichten von Providernamen getrennt. Eine eingeschränkte Pattern-Grammatik und providerabhängige Writer-Projektionen machen Unterschiede explizit. Ein Review des Umbaus fand zunächst optionale Bindungsfelder, bei denen Resume Vergleiche überspringen konnte. Sie wurden verpflichtend; auch die aufgelöste Binary-Identität wird gebunden. [1, 5]

Der vorgelagerte Antigravity-PoC zeigt, warum Live-Proben nötig sind: Eine Kombination aus `prefixItems` und `items: false` wurde abgewiesen. Bei einem anderen Versuch gab es trotz Exitcode 0 keine verwertbare strukturierte Antwort. Ein positiver Einzelfall unter angepasstem Schema war deshalb noch keine Adapterfreigabe. Transporterfolg, Formvertrag, Domainvertrag und Rollenfähigkeit sind getrennte Nachweise. [4, 5]

PORTABILITÄT ALS ÜBERPRÜFBARE EIGENSCHAFT

Neue Kombinationen brauchen Prüfungen für Vertrag, Transport, Rechte, Fehlerwege und die konkrete Rolle. Ein Schema-PoC ersetzt weder Isolation noch einen vollständigen Orchestratorlauf.

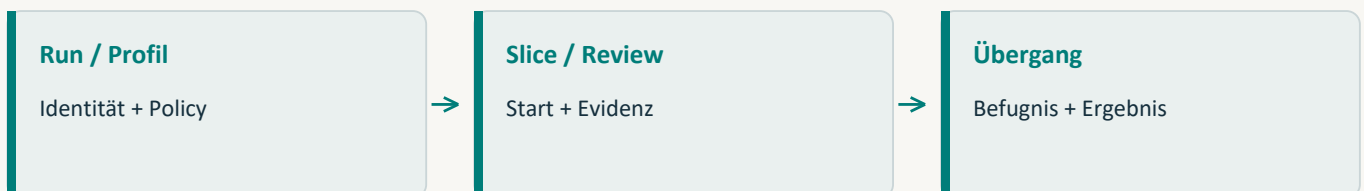
07

PERSISTENZ UND BINDUNGEN

Records tragen die Autorität

Der Protokollmodus heißt weiterhin „structured-v2“. Artefakt-Records und neutrale Provider-Nachrichten verwenden v3-Schemas. Protokollmodus, Schema und Reducer sind getrennte Versionsebenen.

Validierte append-only Records gemäß `orchestrator-artifact-v3` (`schema_version: "3"`) bilden die technische Quelle der repräsentierten Fakten. `RunIdentity` und `RunProfile` binden Auftrag, Branchbasis, Modus und Rollenprofile. `SliceBoundary` hält den unveränderlichen Start eines Slices fest. Ergebnisse, Befunde, Validierung und Übergänge verweisen auf ihren Kontext. Der installierte Reducer rekonstruiert daraus den Betriebszustand. [1]



Hash- und Vorgängerbindungen, hier als lineares Schema vereinfacht

ABB. 05 Die Grafik erklärt Abhängigkeiten; sie bildet nicht jedes Recordformat oder jede Vorgängerrelation ab.

Was Hashes leisten

Der Repository-Fingerprint bindet die Prüfung an einen konkreten Stand. Outputdigests binden gespeicherte Evidenz; Inhalts- und Vorgängerbindungen machen Inkonsistenzen innerhalb der prüfbaren Historie erkennbar. Hashes beweisen jedoch weder die Wahrheit eines Reviewerurteils noch schützen sie allein gegen einen Angreifer, der alle Records und Referenzen neu schreiben kann. Dafür wäre eine zusätzliche Vertrauensgrenze nötig.

Caches sind entbehrlich, Fakten nicht

`state.json`, Checkpoints und `head.json` sind abgeleitete Hilfen. Ein fehlender oder unpassender Cache kann aus einer gültigen Kette neu entstehen. Eine beschädigte oder inkompatible Kette darf dagegen nicht durch geratenen Zustand oder Markdown-Einträge repariert werden. Ältere Protokolle und fremde Reducersemantik werden abgewiesen. Append-only ist dabei eine durchgesetzte Softwarekonvention auf Dateien, kein unveränderliches Speichermedium. [1]

08

RECOVERY

Weiter nur bei klarem Zustand

Ein Neustart darf Wiederholung ermöglichen. Er darf eine unbekannte Außenwirkung nicht als erfolgreich oder folgenlos erfinden.

Git-Commit, Providerstart, autoritatives Schreiben und Queue-Bewegung werden durch stabile Intent-/Result-Paare erfasst. Bei offenem Intent darf Resume die betreffende Außenwirkung gezielt abgleichen. Ist sie nachweislich erfolgt, wird das Ergebnis nachvollzogen; ist sie nachweislich nicht erfolgt, kann die erlaubte Wiederholung folgen. Ein unklarer Ausgang hält den Lauf an. [1]

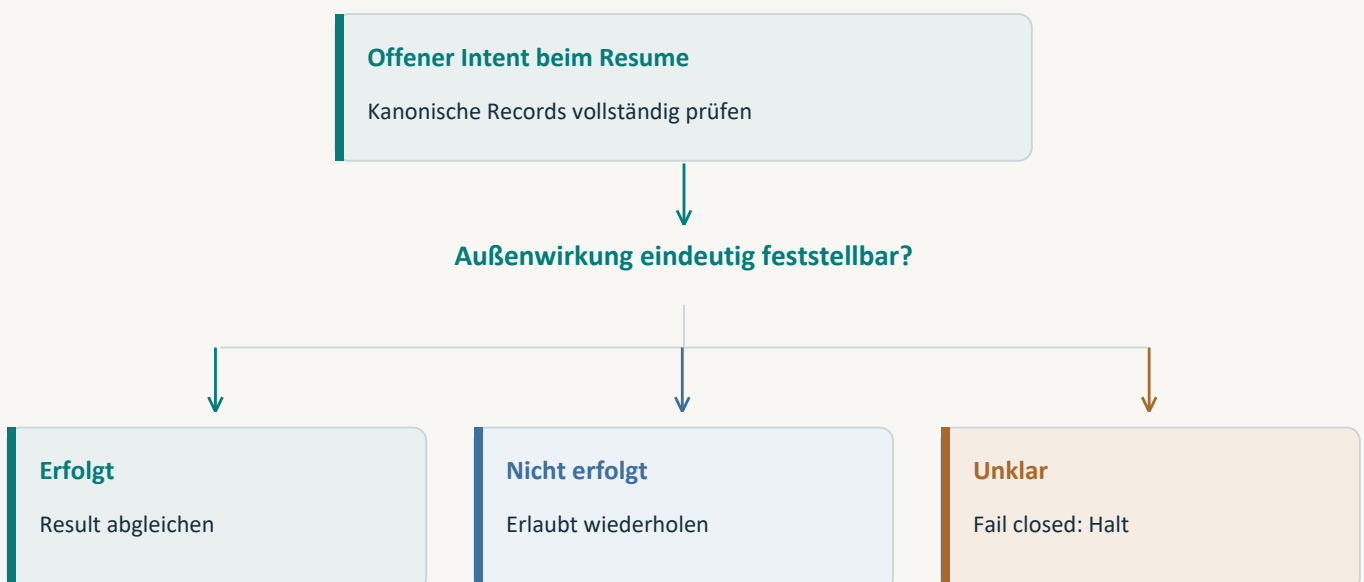


ABB. 06 Recovery-Entscheidung, schematisch. Für jeden Effekt gelten dessen eigene Abgleichregeln; „noch einmal starten“ ist kein allgemeiner Ersatz.

Auch die Initialisierung ist streng begrenzt: Nur ein exakt zulässiger unvollständiger Baseline-Präfix darf vor jeder Außenwirkung vervollständigt werden. Fehlende Fakten in einem bereits aktiven Lauf werden nicht nachgetragen. So bleibt ein gut rekonstruierbarer Cacheverlust von einem nicht sicher rekonstruierbaren Faktenverlust getrennt. [1]

Diese Architektur verspricht keine universelle Exactly-once-Ausführung beliebiger Außenwirkungen. Sie schafft explizite Wiederaufnahmegrenzen und stoppt bei Unsicherheit. Der reale Abbruch auf Seite 10 belegt einen konkreten SIGINT-/Resume-Pfad; der gesonderte Crash-Testbestand prüft weitere definierte Grenzen. Beides bleibt an getestete Fälle und Versionen gebunden.

Ein echter Auftrag mit Abbruch

Auto-Optimize erhält einen eigenen Tab. Der Lauf verbindet normale Entwicklungsarbeit, eine absichtliche Unterbrechung und eine notwendige Korrektur.

Zielsystem: RuhestandsApp, eine bestehende JavaScript-Anwendung mit Tauri-Desktopintegration. Ihre Regressionstestsuite meldete im abschließenden Lauf von `npm test` **20.196 bestandene Assertions**. [3, 6]



Auftrag: vorhandenen Tab-Mechanismus und IDs erhalten; keine Änderung an Engine, Workern oder Optimierungslogik. Layouttests und Hilfe sollten den neuen Ort abbilden. [3, 6]

- 14:23

●

Start

Ein Implementierungsslice auf der RuhestandsApp.
- 14:28

●

Validierung rot

Ein bestehender Test erwartet weiterhin vier Tabs.
- 14:29

●

Abbruch und Resume

SIGINT um 14:29:21; Resume ab 14:29:53.
- 14:30

●

Review verweigert

R-01 fordert Klärung und Behebung der roten Suite.
- 14:32-34

●

Vertrag und Scope

Unzulässige Antwort verworfen; zusätzliche Testdatei angefordert.
- 14:37-38

●

Grün und abgeschlossen

npm test PASS; R-01 geschlossen; Gesamtreview ohne neue Findings.

Zeiten: 28.09.2026, Europe/Berlin. Beim Abbruch lief ein Claude-Review. Nach zwei Sekunden war der Orchestrator beendet; die anschließende Prozesskontrolle zeigte auch keinen verbleibenden Providerprozess. Resume kennzeichnete den alten Versuch als fehlgeschlagen und startete Versuch 2 derselben logischen Operation. [3]

Nachweisgrenze: Der separate Browser-Smoke war nicht Bestandteil der attestierten npm-test-Matrix. Das Abschlussreview hielt die verbleibende Unsicherheit zur Laufzeitinitialisierung fest. Der später erfolgte Merge erweitert diesen Testnachweis nicht.

10

UMFANG UND ERFAHRUNGEN

Zahlen mit benanntem Nenner

Der Bestand ist groß genug für reale Integrationsprobleme. Zeilenzahlen messen Umfang und Pflegeaufwand, weder Qualität noch Nutzen.

150.729

Python-Zeilen gesamt

1,41 : 1

Testcode / src-Code

2.884 + 39

bestandene Tests¹

Gezählter Bestand	Dateien	Physische Zeilen
Produktivcode unter src/	80	61.441
Testcode unter tests/	113	86.755
Hilfsskripte unter scripts/	3	2.533

Gezählt werden versionierte Python-Dateien im festgehaltenen Git-Baum, einschließlich Leerzeilen, Kommentaren und Docstrings. 86.755 / 61.441 = 1,412. Die Skripte stehen separat; mit ihnen im Produktivnenner beträgt das Verhältnis 1,36 : 1. Es handelt sich nicht um Testabdeckung. [1, 7]

Testnachweise und historische Einsätze

¹ Die gespeicherten Phase-A-Protokolle melden 2.884 bestandene Tests in der Default-Suite und 39 bestandene Crash-Harness-Tests in einem separaten Lauf. Diese Zahlen wurden für das Papier aus vorhandenen Logs gelesen; es wurde keine neue Validierungsattestierung erzeugt. Der reale Funktionsnachweis referenziert den abgenommenen Feature-HEAD ausdrücklich. [3, 7]

Eine frühere, fest besetzte Orchestratorversion implementierte eine vollständige Flutter-Anwendung aus einer vorgegebenen Spezifikation. Dabei waren zwei bis drei Rückfragen erforderlich. Das anonymisierte Beispiel veranschaulicht einen früheren Entwicklungseinsatz außerhalb der Phase-A-Abnahme; Projektname, Domäne und Spezifikationsdetails bleiben unveröffentlicht. [8]

Weitere Einsätze umfassten Fehlerbehebung in der RuhestandsApp und Bootstrap-Arbeiten am Orchestrator. Die Phase-A-Entwicklung zeigt zugleich den Aufwand: Übersehene Bindungen und ein Fehler beim Schreiben großer Prompts wurden im Review erkannt und vor der Abnahme korrigiert. Selbstverbesserung braucht dieselben Prüfgrenzen wie Änderungen an einer fremden Anwendung. [5, 8]

11

BETRIEBSMODELL

Zeit kann Budget ersetzen

Quota-Grenzen sind Teil des Betriebs: Der Orchestrator kann auf erneute Verfügbarkeit warten und die Arbeit anschließend fortsetzen.

Die Größe eines Providerkontingents bestimmt, wie viel Arbeit zwischen zwei Pausen möglich ist. Erreicht ein unterstützter Provider sein Nutzungslimit, hält der Orchestrator den Ablauf an. Aus einer eindeutig erkannten Reset-Zeit plant er die Wiederaufnahme und wartet bis zu diesem Zeitpunkt zuzüglich Sicherheitsabstand. Anschließend setzt er die unterbrochene Arbeit fort, sofern die Resume-Prüfungen erfüllt sind. So können auch kleine Abokontingente für längere Aufgaben genutzt werden. [1]

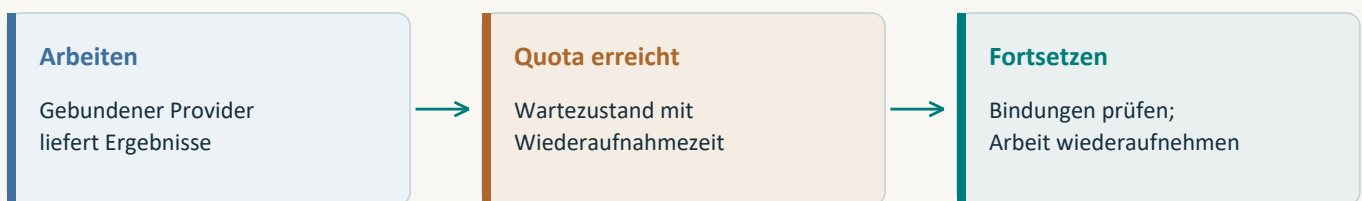


ABB. 07 Schematischer Quota-Pfad. Während der Pause bleiben Arbeitsstand, Rollenprofile und Prüfregeln erhalten.

Die Quota-Erkennung wertet die Meldungen der unterstützten Provider aus. Eindeutige Reset-Zeiten werden auf UTC normalisiert; maximale Wartezeit, Sicherheitsabstand und Zahl automatischer Wiederaufnahmen sind konfigurierbar. Fehlt eine eindeutige Reset-Zeit oder ist eine konfigurierte Grenze erreicht, bleibt der Lauf für eine explizite Wiederaufnahme angehalten. Die Kontrollregeln für Review, Validierung und Commit gelten auch nach der Pause unverändert. [1]

Die offiziellen Anbieterunterlagen unterscheiden Zugangswege: Codex unterstützt für lokale Arbeit die Anmeldung über ChatGPT oder einen API-Schlüssel. Claude Code kann Pro-/Max-Kontingente nutzen; Limits werden mit weiterer Claude-Nutzung geteilt. Separat gewählte API-Nutzung wird separat abgerechnet. Diese Wahl betrifft die Authentifizierung und Abrechnung der jeweiligen CLI. Sie bedeutet keinen bereits implementierten direkten HTTP-API-Adapter im Orchestrator. [9, 10]

PRAKTISCHE KONSEQUENZ

Preiswerte Abos werden nutzbar, wenn Aufgaben über mehrere Kontingentfenster laufen dürfen. Wer kürzere Durchlaufzeiten benötigt, kann größere Kontingente oder API-Abrechnung wählen. Die gewählte Zugangsart bestimmt das verfügbare Budget; die Prüfregeln bleiben dieselben.

12

EINORDNUNG UND AUSBLICK

Bekannte Bausteine, eigener Fokus

Der Beitrag liegt in der konkreten Verbindung und Durchsetzung der Verträge. Die Architektur beansprucht keine Erfindung des Vier-Augen-Prinzips oder von Event Sourcing.

ESAA beschreibt Event Sourcing für autonome Agenten und umfasst eine Implementierung sowie zwei Fallstudien. Gemeinsam sind die explizite Ereignishistorie, der rekonstruierbare Zustand und die Trennung von Vorgängen und Projektionen. Mit ESAA-Core existiert zudem ein Softwareprojekt. Die verwandten Ansätze haben damit sowohl eine konzeptionelle als auch eine praktische Grundlage. [11]

Der praktische Schwerpunkt des **Dual-Agent-Orchestrators** liegt in den Betriebsgrenzen eines Entwicklungsworkflows auf Git-Repositories: heterogene Provider-JSON-Dialekte, reviewer-eigene Findings, fingerprintgebundene Validierung sowie kontrollierte Git- und Dateisystemwirkungen. Prozessunterbrechungen und Resume müssen diese Bindungen unter Linux beziehungsweise WSL 2 erhalten. Dafür verbindet Phase A konkrete Mechanismen mit Tests und einem realen Abnahmelauf. [1, 3, 5]

LangGraph dokumentiert Checkpoints, Zustandsinspektion und Wiederaufnahme. **Anthropic** beschreibt das Evaluator-Optimizer-Muster als wiederholte Erzeugung und Bewertung. Diese Einordnung benennt verwandte Bausteine; ein Qualitätsvorsprung gegenüber ESAA oder anderen Ansätzen wurde nicht vergleichend gemessen. [11, 12, 13]

Was offen bleibt

Die reale Abnahme umfasst einen überschaubaren UI-Umbau mit einer aktuellen Providerbesetzung. Für Erfolgsquoten, mittlere Korrekturrunden, Warteanteil und Kosten je abgeschlossener Aufgabe fehlt eine ausreichend breite Messreihe. Mehr Testcode ist kein Ersatz dafür. Auch Herstellertrennung garantiert keine statistische Unabhängigkeit: Modelle können dieselben Annahmen teilen oder denselben Fehler übersehen.

Die nächste Providerintegration muss Rechte und Konfigurationsisolation praktisch angreifen, Schema- und Transportgrenzen nachweisen und die Zielrolle in vorab festgelegten Fällen prüfen. Die Phase-B-Planung fordert dabei denselben Isolationsmaßstab auch für den bestehenden Claude-Reviewer. Ein erfolgreicher PoC oder das bloße Entfernen eines Provider-Pins genügt nicht. [4]

DIE NÄCHSTE SINNVOLLE MESSREIHE

Mehrere reale Aufgaben je zugelassener Topologie: Abschlussstatus, Findings, Korrekturen, Vertragsfehler, aktive Laufzeit, Quota-Wartezeit und tatsächlicher Verbrauch. Vorab definierte Fälle und vollständig sichtbare Fehlversuche machen die Ergebnisse vergleichbar.

13

QUELLEN UND METHODE

Nachvollziehbar bis zum Stand

Quellenstand: 28. September 2026. Repositorybezüge sind auf den beschriebenen Stand fixiert; externe Dokumentation kann sich später ändern.

- [1] [Phase-A-Code und Rollenvertrag](#). Abgenommener Stand vom 28.09.2026: AGENTS.md, src/, schemas/ und README.md. Versionbindung und Nachweise: docs/reference/whitepaper-phase-a-evidenz.md im Repository.
- [2] [Rollenfreigaben und Konfiguration](#). Dazu orchestrator.toml: Slots, Profile, Workflowlimits.
- [3] Lokales Abnahmepaket: Start-/Endmetadaten, Abbruchprotokoll, Resume-Log und strukturierte Records. Der gespeicherte npm-test-Output meldet 20.196 bestandene Assertions. Testoutput mitveröffentlicht; übriges Rohpaket lokal eingesehen.
- [4] Lokale Planung „Task B - Antigravity als zusätzlicher Reviewer“ und „Task C - Freie Rollenbesetzung nach Kombination qualifizieren“, 27.09.2026. Eingesehene Planungsunterlagen, keine bereits implementierte Unterstützung.
- [5] Lokaler Antigravity-PoC-Bericht vom 27.09.2026 sowie Entscheidungsprotokoll und Branchreview der Phase-A-Steuerung. Entwicklungsbefunde, kein Leistungsbenchmark. Originalunterlagen nicht mitveröffentlicht.
- [6] [RuhestandsApp: Implementierung des neuen Tabs](#). Quelländerungen, Tests und Dokumentation zur Trennung von Parameter-Sweep und Auto-Optimize.
- [7] Eigene Bestandszählung aus dem fixierten Git-Baum: git archive, nur *.py in src/, tests/, scripts/, physische Zeilen via splitlines(). Mitveröffentlichte task-a-default.log / task-a-crash.log: 2.884 / 39 PASS. Fundorte und Prüfsummen im Evidenzindex; keine neue Attestierung.
- [8] Projektgeschichte und frühere Anwendungseinsätze: Markdown-Kommunikation, anonymisierte Flutter-Implementierung, Fehlerbehebung und Bootstrap. Beschreibende Fallbeispiele außerhalb der formalen Phase-A-Abnahme.
- [9] [OpenAI: Authentication](#). Offizielle Beschreibung von Abo- und API-Anmeldung für Codex; Abruf 28.09.2026.
- [10] [Anthropic: Claude Code mit Pro oder Max](#). Offizielle Beschreibung von gemeinsamem Kontingent, Reset und separater API-Abrechnung; Abruf 28.09.2026.
- [11] [ESAA: Event Sourcing for Autonomous Agents in LLM-Based Software Engineering](#). Elzo Brito dos Santos Filho, 2026, arXiv:2602.23193. Preprint mit Implementierung und Fallstudien; [ESAA-Core](#) als Softwareprojekt.
- [12] [LangGraph: Persistence](#). Offizielle Dokumentation zu Checkpoints und Wiederaufnahme; Abruf 28.09.2026.
- [13] [Anthropic: Building effective agents](#). Evaluator-Optimizer als Workflowmuster; Abruf 28.09.2026.
- [14] [Google: Antigravity CLI](#). Offizielle Beschreibung des terminalbasierten Agentenwerkzeugs und des Aufrufs agy; Abruf 28.09.2026.